

Lo que aprendimos

Retrospectiva y lecciones de construir Bodymetria
Bodymetria · 12/07/2026

Una retrospectiva del proyecto: no la referencia técnica (esa está en `documentacion-tecnica.pdf`), sino las **lecciones** que dejó construirlo, tanto técnicas como de proceso. Está escrita para recordar POR QUÉ tomamos cada camino.

1. Empezar por la decisión que cambia todo

Antes de escribir una línea, la primera pregunta fue la tecnología. Elegir **Kotlin + Jetpack Compose** (nativo) en vez de multiplataforma no fue casual: la fase 2 necesita APIs nativas (uso de apps, sensores de salud), y esa decisión temprana evita reescribir después. Lección: **las decisiones de arquitectura cuestan barato al principio y carísimo tarde**. Lo mismo con iOS — decidir "todavía no, pero con Compose la puerta queda abierta" es una decisión válida.

2. Construir en capas separadas paga

Dividir en **datos / lógica / UI** (y en el sitio, datos / cálculo / páginas) hizo que cada cambio fuera local. El mejor ejemplo: las recetas no guardan calorías "a mano", referencian la base de alimentos; cuando corregimos un alimento, **todas** las recetas que lo usan se arreglan solas. Una sola fuente de verdad ahorra bugs.

3. Alcanzar la meta con datos reales, no con supuestos

Para las calorías usamos **Mifflin-St Jeor** (el índice estándar) y le sumamos el gasto de ejercicio REAL que registra el usuario, no un factor inventado. Y cuando el menú no cuadraba con la meta, el problema no era el cálculo sino el **redondeo** de porciones a media ración: al pasar a un factor continuo, cada día cae en el objetivo. Lección: cuando algo "no da", sospecha primero de los redondeos y los bordes, no del promedio.

4. Honestidad sobre exactitud aparente

Varias veces preferimos decir la verdad incómoda antes que un número bonito:

- La **vitamina D** casi no se obtiene de la comida — se marca "del sol", no como una falla del menú.
- Los **ciclos de sueño de 90 min** son un heurístico débil; dormir 9 h no es "lo ideal" (el óptimo es 7-8 h). Quitamos el falso "ideal".
- En **suplementos**, distinguimos lo que tiene evidencia sólida (creatina) de lo prescindible (beta-alanina, citrulina), citando papers.

Lección: la confianza se gana admitiendo los límites, no ocultándolos.

5. Seguridad como diseño, no como parche

La app no pide permisos ni usa red: el sandbox de Android ya protege los datos. El punto débil era el **import de JSON**; en vez de confiar en el archivo, lo validamos (rangos, fechas, tamaños) y mostramos una vista previa antes de tocar nada. Regla general: **los datos externos son entrada hostil hasta demostrar lo contrario**, y toda acción destructiva se confirma.

6. Las trampas del entorno se documentan

Esta máquina tenía peculiaridades que costaron tiempo hasta entenderlas, y por eso quedaron escritas:

- **TLS interceptado en la red:** la JVM necesita el almacén de certificados de Windows y `curl` necesita `--ssl-no-revoke`; si algo "no descarga", casi siempre es esto.
- **Instaladores que se cuelgan:** cuando el gestor de paquetes fallaba, bajar el zip directo y colocarlo a mano fue más confiable.
- **Virtualización de AppData** (apps empaquetadas MSIX): lo instalado podía quedar en una carpeta oculta del paquete e invisible para el resto del sistema; se resolvió moviendo el SDK al disco real.

Lección: **anota la trampa la primera vez**; la segunda vez que aparezca la resuelves en minutos.

7. Verificar de verdad, no "debería funcionar"

Montamos un **emulador** y probamos la app tocándola (registrar peso, ver la meta, crear una métrica médica); el sitio lo revisamos en el navegador real (consola sin errores, valores correctos, micros que cumplen). Encontramos así cosas que "en teoría" estaban bien pero fallaban en pantalla. Lección: **ejecutar y mirar** vale más que suponer.

8. Iterar con el usuario, en pasos chicos

El proyecto creció por rondas cortas: una función, verificar, ajustar la redacción, seguir. Muchos cambios salieron de "no me gustó cómo se ve" o "esto no cuadra" — y esos ajustes finos (el icono, el tono de un texto, un platillo más) son los que hacen que se sienta propio. Lección: **entregar seguido y escuchar** supera a planear todo de una vez.

9. Publicar es su propio proyecto

Poner el sitio en línea trajo su propia lista de aprendizajes:

- **Netlify** solo hace auto-deploy si está conectado a un repo; arrastrar la carpeta (drag-drop) es un despliegue suelto que no se actualiza.
- La config (`netlify.toml`) se lee desde la **raíz** del repo, y `publish` es relativo a base; equivocarse da un "page not found".
- El **APK** (17 MB) no debe vivir en el repo: se publica como **release** de GitHub y el botón apunta a "latest".

Lección: distribución y desarrollo son fases distintas, cada una con sus reglas.

10. Lo que queda

El proyecto está vivo: faltan la captura automática (uso de apps, salud), la publicación en Play Store (keystore, AAB, política de privacidad, prueba cerrada) y, si el mercado lo pide, iOS. Pero la base es sólida y —sobre todo— **está entendida y documentada**, que era medio objetivo desde el principio.